

Claude Code Codex Strongest

完整安装、配置、使用、排错与维护指南

Liu Jiarui

2026 年 6 月

目录

阅读说明	vii
第一章 项目总览	1
1.1 项目名称与仓库	1
1.2 一句话解释	1
1.3 适合谁	1
1.4 核心组成	2
第二章 快速安装	3
2.1 Windows 一键安装	3
2.1.1 Windows 一行命令	3
2.1.2 Windows 本地仓库运行	3
2.2 macOS 一键安装	4
2.2.1 macOS 一行命令	4
2.2.2 macOS 本地仓库运行	4
第三章 安装器细节	5
3.1 Windows 安装器流程	5
3.2 macOS 安装器流程	5
3.3 Bootstrap 脚本	6
3.3.1 固定版本或分支	6
第四章 安装内容清单	7
4.1 基础工具	7
4.2 CLI 与 VS Code 扩展	7
第五章 为什么是 Strongest：完整配置资产逐项拆解	9
5.1 Strongest 的判定标准	9
5.2 CLAUDE.md：全局入口，不是普通 README	9
5.3 小白图文流程：从中转站到 cc-switch	10
5.3.1 中转站控制台：URL、Key、模型名	10
5.3.2 cc-switch：添加 provider、模型映射、路由代理	10
5.4 33 个 Skills：把专家方法固化成可触发流程	13
5.5 22 个 Agents：把复杂任务交给专门角色	16

5.6	25 个 Commands: 把 workflow 变成一条入口	17
5.7	Hooks: 把安全、质量和上下文自动化	17
5.8	settings.json: 生命周期编排核心	18
5.8.1	环境变量设计	18
5.8.2	statusLine 与 permissions	19
5.8.3	生命周期注册矩阵	19
5.9	为什么这些组合起来才是 Strongest	19
第六章	Codex 配置	21
6.1	目录与模板	21
6.2	不会复制什么	21
6.3	Codex 登录	21
第七章	MCP Servers	23
7.1	Windows 与 macOS 的命令差异	23
7.2	写入策略	24
第八章	API Key 与模型配置	25
8.1	Claude Code 侧: cc-switch	25
8.2	Codex 侧: codex login	25
8.3	模型名与中转站	25
第九章	常用操作	27
9.1	验证安装	27
9.2	Reset 干净重装	27
9.3	非交互安装	27
第十章	排错指南	29
10.1	GitHub 下载失败	29
10.2	winget msstore 证书错误	29
10.3	VS Code 扩展安装出现 Node DeprecationWarning	29
10.4	正在 Claude Code 内运行安装器导致.claude 被占用	29
10.5	Codex doctor WebSocket warning	30
10.6	codex 命令找不到	30
第十一章	安全模型	31
11.1	不迁移凭证	31
11.2	Git 安全	31
第十二章	维护与发布	33
12.1	仓库重命名	33
12.2	提交前检查	33
12.3	发布检查表	33

12.4 Release 与 CI 边界	34
12.5 容易写错或理解错的点	34
第十三章 附录：关键命令速查	35
13.1 安装	35
13.2 验证	35
13.3 登录	35

阅读说明

本指南面向 **Claude Code Codex Strongest / claude-code-codex-strongest** 项目的使用者、维护者和二次开发者。它不仅说明如何双击一键安装，也解释安装器到底做了什么、每个配置目录放什么、Claude Code 与 Codex 的边界在哪里、如何验证安装是否成功，以及常见故障如何定位。

说明

项目定位: 这是一个同时覆盖 **Claude Code** 与 **Codex CLI** 的开箱即用配置包。它把 Claude Code 的 skills、agents、commands、hooks、MCPs 和 Codex 的安全模板配置整合到一个跨平台一键安装器中。

注意

安全边界: 安装器不会写入 Codex token，不会复制 `auth.json`，不会把 Claude 登录态迁移给 Codex。Codex 登录必须由用户安装后手动执行 `codex login`。

第一章 项目总览

1.1 项目名称与仓库

项目展示名为 **Claude Code Codex Strongest**，仓库名为 **claude-code-codex-strongest**。
GitHub 地址为：

```
https://github.com/liujiarui0918/claude-code-codex-strongest
```

本项目原本是 Claude Code Strongest，后来扩展为同时安装和配置 Claude Code 与 Codex，因此命名为 Claude Code Codex Strongest。

1.2 一句话解释

这是一个面向 Windows 和 macOS 的双栈 AI 编程环境一键安装包：

- 安装 **Claude Code CLI** 与 VS Code 插件 `anthropic.claude-code`；
- 安装 **Codex CLI** 与 VS Code 插件 `openai.chatgpt`；
- 部署 `~/.claude` 全套 workflows 配置；
- 部署 `~/.codex` 安全模板；
- 配置 8 个 MCP server；
- 安装 VS Code、Git、Node.js、uv、cc-switch 等基础工具。

1.3 适合谁

- 从零开始使用 Claude Code / Codex 的新手；
- 想要完整工程实践配置而不是空白默认配置的中级用户；
- 需要统一 Windows / macOS 开发环境的团队；
- 想 fork 后定制自己 AI 编程工作流的高级用户。

1.4 核心组成

组成	作用
<code>install-windows.bat</code>	Windows 用户双击入口, 下载 GitHub 最新 main 包并调用 PowerShell 安装器。
<code>install-macos.command</code>	macOS 用户双击入口, 下载 GitHub 最新 main 包并调用 Bash 安装器。
<code>install/install-windows.ps1</code>	Windows 主安装器, 负责安装依赖、部署 Claude/Codex 配置、验证安装。
<code>install/install-macos.sh</code>	macOS 主安装器, 负责 Homebrew、VS Code、CLI、扩展与配置部署。
<code>~/.claude</code>	Claude Code 全局配置目录, 包含 skills、agents、commands、hooks、docs 等。
<code>~/.codex</code>	Codex 全局配置目录, 包含 AGENTS.md、config.toml、.gitignore 等安全模板。
<code>~/.claude.json</code>	Claude Code MCP server 配置文件。
<code>.codex/</code>	仓库内 Codex 安全模板, 安装器会复制其中安全文件到用户 <code>~/.codex</code> 。

第二章 快速安装

2.1 Windows 一键安装

推荐方式：从 Releases 页面下载 `install-windows.bat` 并双击运行。

```
https://github.com/liujiarui0918/claude-code-codex-strongest/releases/latest
```

安装器会自动完成：

1. 安装或确认 VS Code;
2. 创建/刷新 Windows 桌面 Visual Studio Code.lnk;
3. 安装 Git、Node.js、uv;
4. 安装 Claude Code CLI: @anthropic-ai/claude-code;
5. 安装 Codex CLI: @openai/codex;
6. 安装 VS Code 扩展: anthropic.claude-code、openai.chatgpt、中文语言包、Office Viewer;
7. 安装 cc-switch;
8. 部署 ~/.claude;
9. 部署 ~/.codex 安全模板;
10. 写入 ~/.claude.json 的 8 个 MCP server;
11. 运行安装验证。

2.1.1 Windows 一行命令

```
irm https://raw.githubusercontent.com/liujiarui0918/claude-code-codex-strongest/main/install/bootstrap.ps1 | iex
```

2.1.2 Windows 本地仓库运行

```
git clone https://github.com/liujiarui0918/claude-code-codex-strongest.git
cd claude-code-codex-strongest
.\install\install-windows.ps1
```

2.2 macOS 一键安装

推荐方式：从 Releases 页面下载 `install-macos.command` 并双击运行。若 macOS 阻止运行，可右键选择“打开”。

2.2.1 macOS 一行命令

```
curl -fsSL https://raw.githubusercontent.com/liujiarui0918/claude-code-codex-strongest/main/install/bootstrap.sh | bash
```

2.2.2 macOS 本地仓库运行

```
git clone https://github.com/liujiarui0918/claude-code-codex-strongest.git
cd claude-code-codex-strongest
bash install/install-macos.sh
```

第三章 安装器细节

3.1 Windows 安装器流程

Windows 主安装器为 `install/install-windows.ps1`，要求 PowerShell 5.1 或更高版本。它的主要阶段如下：

阶段	说明
Phase 1	通过 <code>winget / npm</code> 安装 VS Code、Git、Node.js、uv、Claude Code CLI、Codex CLI、VS Code 扩展，并创建 VS Code 桌面快捷方式。
Phase 2	部署仓库文件到 <code>~/.claude</code> 。若目录被当前 Claude Code 会话占用，安装器会 fallback 到 robocopy 复制备份，而不是直接失败。
Phase 3	从 <code>settings.template.json</code> 渲染 <code>settings.json</code> 。默认不写 API key，除非用户显式传入参数。
Phase 4	将 8 个 MCP server 写入 <code>~/.claude.json</code> 。
Codex	将仓库 <code>.codex</code> 下的安全模板复制到 <code>~/.codex</code> ，只复制 <code>AGENTS.md</code> 、 <code>config.toml</code> 、 <code>.gitignore</code> 、 <code>README.md</code> 。
Phase 5	验证 <code>settings</code> 、 <code>docs</code> 、 <code>skills</code> 、 <code>agents</code> 、 <code>commands</code> 、 <code>hooks</code> 、Codex 文件、VS Code shortcut、VS Code 扩展、MCP server 数量。

3.2 macOS 安装器流程

macOS 主安装器为 `install/install-macos.sh`。它会：

- 安装 Homebrew（若不存在）；
- 安装 VS Code、Git、Node.js、PowerShell 7；
- 安装 uv；
- 安装 Claude Code CLI 与 Codex CLI；
- 确保 `code` 命令可用；
- 安装四个 VS Code 扩展；
- 部署 `~/.claude`；
- 渲染 Claude Code settings；

- 写入 MCP server;
- 调用 `deploy_codex_config` 部署 Codex 模板;
- 验证安装。

3.3 Bootstrap 脚本

`install/bootstrap.ps1` 与 `install/bootstrap.sh` 是“无需 clone”的远程入口。它们会下载 GitHub main 分支压缩包，解压 `claude-code-codex-strongest-*` 目录，再调用平台主安装器。

3.3.1 固定版本或分支

两个 bootstrap 都支持通过 `CCS_REF` 固定版本、分支或 tag。若值形如版本号或 tag，脚本会走 GitHub tags；否则走 heads。

Windows PowerShell:

```
$env:CCS_REF = 'v1.0.0'
irm https://raw.githubusercontent.com/liujiarui0918/claude-code-codex-strongest/main/
install/bootstrap.ps1 | iex
```

macOS / Linux:

```
CCS_REF=v1.0.0 curl -fsSL https://raw.githubusercontent.com/liujiarui0918/claude-code-
codex-strongest/main/install/bootstrap.sh | bash
```

第四章 安装内容清单

4.1 基础工具

工具	Windows 渠道	macOS 渠道
VS Code	winget Microsoft.VisualStudioCode	brew cask visual-studio-code
Git	winget Git.Git	brew git
Node.js LTS	winget OpenJS.NodeJS	brew node
uv	winget astral-sh.uv	官方安装脚本
PowerShell	Windows 自带 5.1 可用	brew cask powershell
cc-switch	winget farion1231.CC-Switch	brew cask cc-switch

4.2 CLI 与 VS Code 扩展

项目	安装内容
Claude Code CLI	npm 包 @anthropic-ai/claude-code, 命令 claude。
Claude Code VS Code 扩展	扩展 ID anthropic.claude-code。
Codex CLI	npm 包 @openai/codex, 命令 codex。
Codex / ChatGPT VS Code 扩展	扩展 ID openai.chatgpt。
Code 扩展	
中文语言包	扩展 ID MS-CEINTL.vscode-language-pack-zh-hans。
Office Viewer	扩展 ID cweijan.vscode-office。

第五章 为什么是 Strongest：完整配置资产逐项拆解

说明

本章是全书亮点。Claude Code Codex Strongest 的强，不是因为名字里有 Strongest，而是因为它把 Claude Code 与 Codex 两套 CLI、VS Code 插件、cc-switch 模型路由、CLAUDE.md 全局规则、docs 方法文档、33 个 skills、22 个 agents、25 个 commands、16 个 hook 脚本、settings.json 生命周期配置、MCP servers、安全边界和真实验证流程整合为一套可安装、可复用、可审计的工程系统。

5.1 Strongest 的判定标准

一个真正强的 AI 编程配置，不应该只回答“能不能聊天”，而要回答五个工程问题：第一，普通用户能否一键装好；第二，复杂任务能否被拆成计划、实现、审查、验证；第三，危险操作和密钥泄漏能否提前拦住；第四，模型、工具、文档、浏览器、Git 能否接入同一条工作流；第五，产物能否被真实命令证明可用。

本项目的答案是：用一键安装器负责环境，用 CLAUDE.md 和 docs 负责规则，用 skills 负责方法，用 agents 负责角色分工，用 commands 负责入口，用 hooks 负责安全和自动化，用 settings.json 负责生命周期注册，用 MCP 负责外部能力，用 Codex 的 AGENTS.md 和 config.toml 负责第二套 CLI 的隔离配置。

5.2 CLAUDE.md：全局入口，不是普通 README

CLAUDE.md 是 Claude Code 每次会话自动加载的高优先级指令入口。它本身很短，但通过模块引用把真正的规则拆到四个文档：

文件	作用
CLAUDE.md	会话总入口。声明全局规则、模块引用、已安装 skills/agents/commands/MCPs/hooks、备份和 doctor 命令。
docs/environment.md	定义 OS、Shell、PowerShell 兼容规则、路径约定、不要用 Unix-only 命令等环境规则。
docs/workflow.md	定义沟通风格、非平凡任务先 plan、Edit 前必须 Read、验证要求、研究搜索策略。

docs/tools.md	定义 skills、subagents、MCP、cron 的使用边界，强调优先用工具而不是重头实现。
docs/safety.md	定义 Git、安全、destructive ops、反模式，明确 reset/force push/删除目录/改系统配置前必须确认。

这种拆分的强点在于：CLAUDE.md 不臃肿，规则又足够完整；每个模块可以单独维护，安装器只要部署整套目录，就能让新会话自动继承工作方法。

5.3 小白图文流程：从中转站到 cc-switch

下面九张图来自 docs/images/，对应 guide-for-beginners.md 的完整上手流程。它们被合并进 PDF，是为了让没有任何基础的用户也能按图操作。

5.3.1 中转站控制台：URL、Key、模型名

图 5.1 展示中转站控制台右侧 API 信息区域，用户需要复制一个接口 URL。注意 URL 末尾不要加 /。



图 5.1: 中转站控制台：右侧 API 信息区域显示多个可用域名。

图 5.2 展示令牌管理页面，用户从这里复制 API Key。Key 只能放在 cc-switch 或脚本参数里，不能提交到 Git，也不能写进文档。

图 5.3 展示模型广场里模型详情页右上角的模型 ID。不同中转站模型名可能不同，必须以控制台显示为准。

图 5.4 展示分组价格。相同模型在不同分组可能价格差很多，选择令牌前要确认分组和倍率。

5.3.2 cc-switch: 添加 provider、模型映射、路由代理

图 5.5 展示 cc-switch 添加 provider 的界面。这里填写供应商名称、API Key 和请求地址。

图 5.6 展示获取模型列表和模型映射。推荐把 Opus 映射到最强模型，Sonnet 映射到日常模型，Haiku 映射到低成本执行模型。

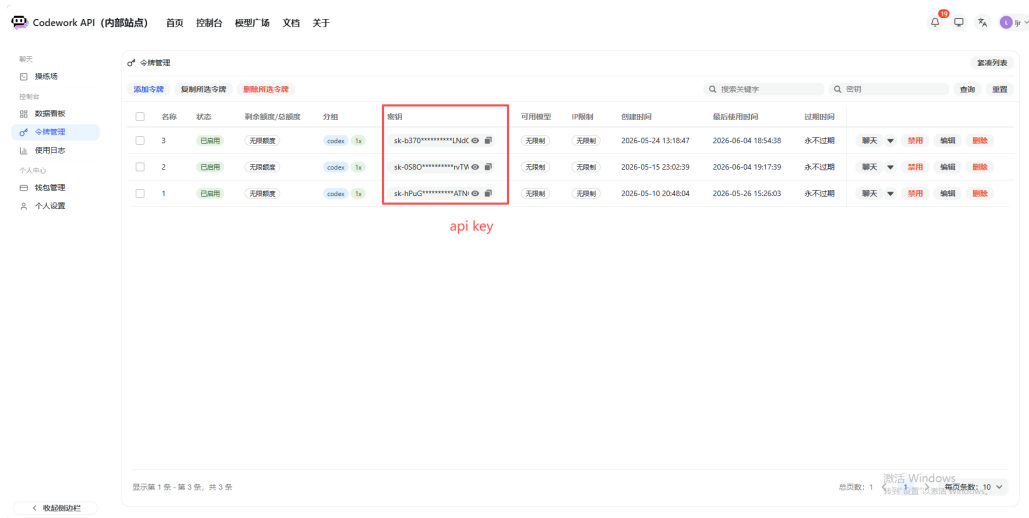


图 5.2: 令牌管理页: 复制 API Key。

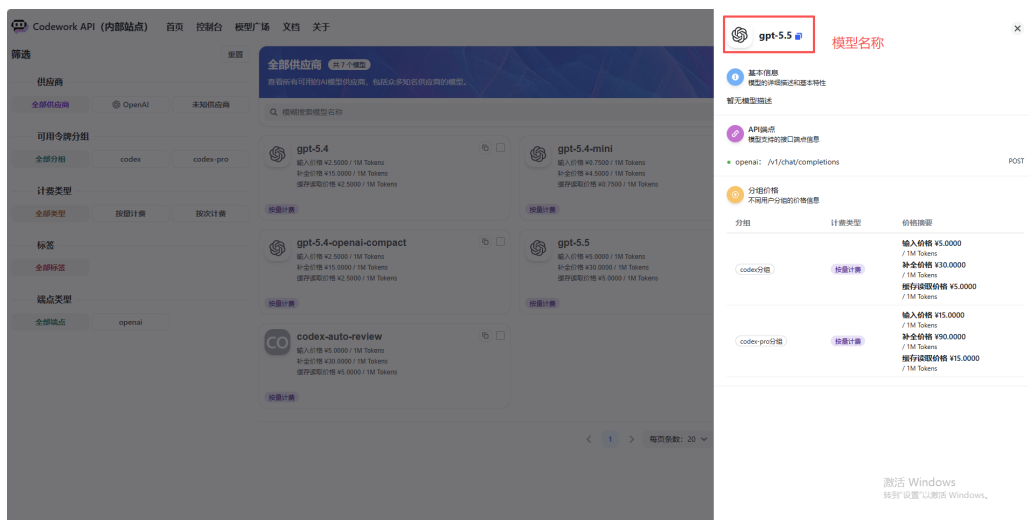


图 5.3: 模型详情: 复制模型 ID。

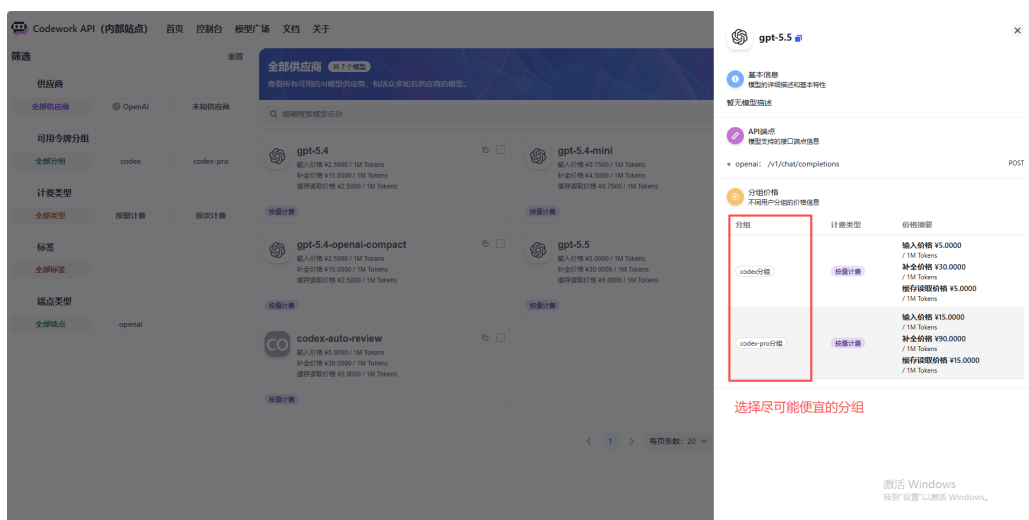


图 5.4: 模型详情: 查看分组价格, 选择成本合适的分组。

← 编辑供应商

供应商名称: anyrouter

备注: 例如: 公司专用账号

官网链接: https://example.com (可选)

API Key:

请求地址: ☐ 完整 URL ☒ 管理 & 测速
https://a-ocnfniawgw.cn-shanghai.fcapp.run

💡 填写兼容 Claude API 的服务端点地址, 不要以斜杠结尾

保存

图 5.5: cc-switch provider 编辑页: 填写 API Key 和请求地址。

← 编辑供应商

✅ 获取到 15 个模型

认证字段: ANTHROPIC_AUTH_TOKEN (默认)

选择写入配置认证环境变量名

模型映射: 一键设置 获取模型列表

显示名称只影响 /model 菜单; 1M 只是给 Claude Code 的上下文能力声明。

模型角色	显示名称	实际请求模型	声明支持 1M
Sonnet	例如 DeepSeek V4 Pro		☐ 1M
Opus	例如 DeepSeek V4 Pro		☐ 1M
Haiku	例如 DeepSeek V4 Pro		

默认兜底模型:

仅在 Claude Code 请求没有明确落到 Sonnet、Opus 或 Haiku 角色时使用; 通常可以留空。

保存

图 5.6: cc-switch 获取模型列表: 为 Opus / Sonnet / Haiku 配置映射。

图 5.7 展示模型端点类型。如果显示 `openai: /v1/chat/completions`，Claude Code 侧需要开启 cc-switch 的 Claude 路由转换。

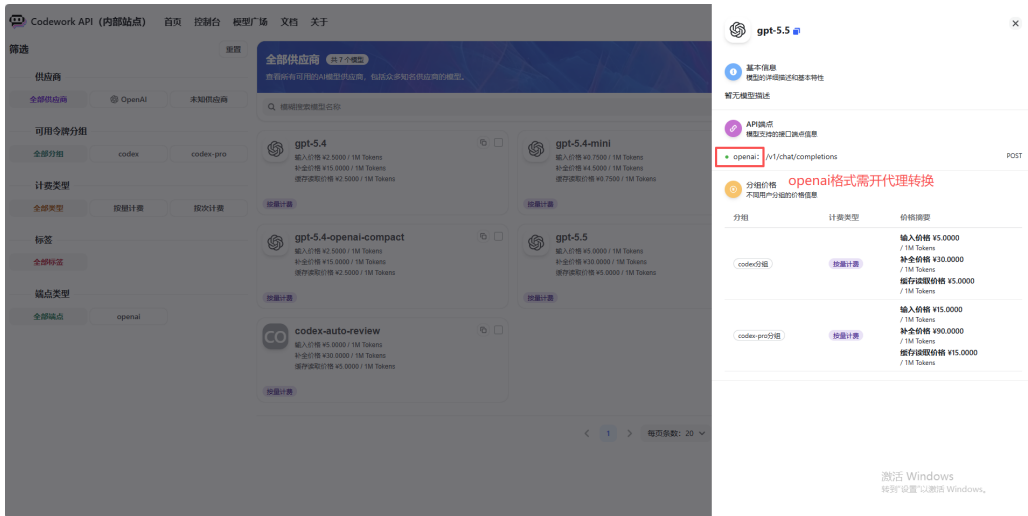


图 5.7: 模型广场：端点显示 openai，说明需要路由转换。

图 5.8 展示路由设置页。需要打开路由总开关，并启用 Claude 路由。



图 5.8: cc-switch 路由设置：打开总开关和 Claude 路由。

图 5.9 展示主界面代理图标变绿，表示路由代理已启用。

5.4 33 个 Skills：把专家方法固化成可触发流程

Skills 是本项目最核心的“方法论层”。它们不是简单提示词，而是给不同任务类型准备的操作规程。下面逐个列出：

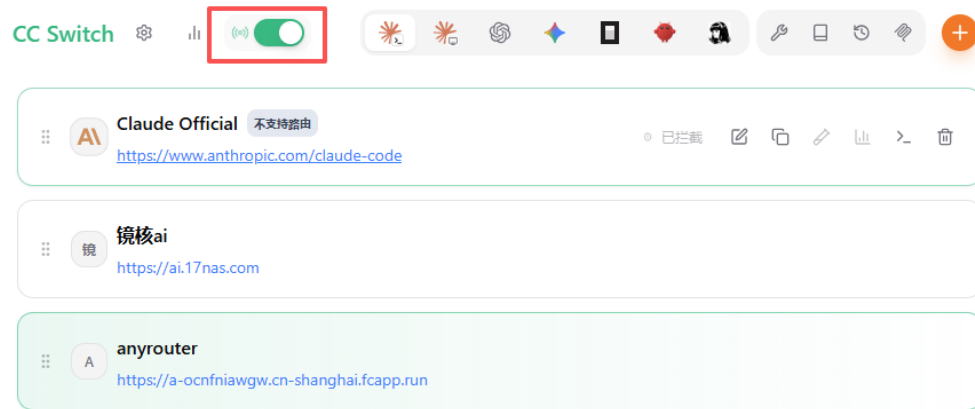


图 5.9: cc-switch 主界面：代理图标变绿表示路由已启用。

Skill	作用	典型场景
mcp-builder	构建新的 MCP server，覆盖 stdio/HTTP、tool schema、错误处理。	当要给 Claude/Codex 增加外部工具能力时使用。
document-handling	处理 docx/xlsx/pptx/pdf 等二进制文档。	生成或读取 Office/PDF 文件，避免用 cat/grep 读二进制。
lean-context	控制上下文体积，只读取必要内容。	长任务、长文件、多轮会话中防止上下文污染。
using-superpowers	会话开始时选择正确 skill。	进入新任务时先判断该用哪个工作方法。
brainstorming	设计前澄清需求、比较 2-3 个方案。	需求不清、架构选择、产品方案讨论。
writing-plans	把方案拆成可验证的执行任务。	准备落地复杂功能或修复前写 PLAN。
executing-plans	按 1-3 个任务一批执行、验证、汇报。	已经有计划文档，需要稳步落地。
subagent-driven-development	复杂任务每个任务派 fresh subagent，并做 spec review。	计划很长、文件很多、主上下文会被污染。
dispatching-parallel-agents	多个独立任务并行分派。	搜索、审查、验证等可以并行完成的工作。

test-driven-development	严格 RED-GREEN-REFACTOR。	明确 bug 复现或明确 API/行为规格。
systematic-debugging	复现、隔离、修复、验证四阶段调试。	任何真实 bug、安装失败、CLI 异常。
verification-before-completion	完成前必须跑真实验证并读输出。	准备声明 done、提交、发布前。
requesting-code-review	交付前自检和请求代码审查。	完成改动但还没交付/commit 前。
receiving-code-review	处理审查反馈，不盲从也不防御。	收到 reviewer 或用户指出问题后。
using-git-worktrees	用 git worktree 隔离 feature。	需要在不污染 main 的情况下做大改动。
finishing-a-development-branch-writing-skills	收尾分支并给出 merge/PR/keep/discard 选项。 编写新 skill 的模板和反模式。	功能完成、测试通过后准备收口。 要把某套工作方法沉淀成 skill。
condition-based-waiting	异步等待必须等条件，不盲目 sleep。	服务启动、CI、后台任务、文件生成。
root-cause-tracing	沿调用链追到根因，不停在症状。	调试和修复阶段防止补丁式修复。
defense-in-depth	关键不变量需要多层防护。	安全、数据删除、权限、凭证保护。
using-context7	查实时库/框架/SDK 文档。	涉及库 API、配置、版本迁移时。
using-deepwiki	理解公开 GitHub repo 的结构和设计。	研究外部仓库、框架、工具实现。
using-sequential-thinking	复杂多步推理和架构选择。	需要分支、修正、逐步推理的问题。
using-playwright	浏览器自动化、截图和 UI 验证。	前端/UI/网页抓取/真实交互验证。
using-mcp-memory	使用知识图谱 memory MCP。	需要实体、关系、观察记录的长期知识。
using-cron	提醒、定时、循环任务。	每 N 分钟检查、每日运行、定时提醒。
using-task-list	判断是否需要任务列表并维护任务。	3+ 步可追踪工作或多人/多 agent 协作。
using-claude-api	Claude API / Anthropic SDK 参考。	写 Anthropic SDK、模型、tool use、缓存、流式。
extracting-from-images	从截图/图片提取文字、布局、意图。	用户贴图、UI 截图、错误截图。

prompt-engineering	编写和改进 LLM prompt。	需要结构化输出、few-shot、评测 prompt。
incremental-context-building	进入陌生大代码库时分层建立理解。	新项目 onboarding、大型代码库探索。
skill-creator	创建新 skill 的标准化流程。	把成熟流程产品化成可复用 skill。
using-memory	使用 auto-memory 系统，判断什么该记。	用户要求记住偏好或项目事实。

5.5 22 个 Agents：把复杂任务交给专门角色

Agents 是“角色层”。当任务需要大量上下文、专项知识或并行处理时，主线程不应该自己吞下所有文件，而应把任务分给专门 agent。

Agent	职责
planner	规划非平凡实现，输出步骤、关键文件、风险。
code-reviewer	代码质量、可维护性、正确性审查，只读。
security-reviewer	OWASP、secrets、注入、鉴权鉴权审查。
tdd-guide	用 TDD 驱动新功能或 bug 修复。
debugger	系统性复现、隔离、修复、验证 bug。
test-runner	运行测试并解释失败。
refactor-assistant	安全重构，保持行为不变。
doc-writer	README、API 文档、变更日志写作。
api-designer	REST/GraphQL/gRPC API schema 设计。
performance-analyst	先 profiling 后优化，定位性能热点。
pr-reviewer	PR 描述与 diff review。
research-agent	多步研究、资料综合。
explore	快速只读代码库探索、定位符号和引用。
build-fixer	修复构建、类型、lint、工具链错误。
typescript-reviewer	TypeScript 专项审查，关注 any、断言、泛型、async。
python-reviewer	Python 专项审查，关注可变默认值、async、类型。
git-operator	高级 Git 操作，如 rebase、cherry-pick、reflog 恢复。
regex-expert	编写、解释、调试正则。
sql-expert	SQL 查询、schema、索引、迁移和 explain。
prompt-engineer	Prompt 设计、结构化输出、反模式。
migration-assistant	语言/框架迁移，如 Python、React、Django。
accessibility-auditor	WCAG、ARIA、键盘、屏幕阅读器、对比度审查。

5.6 25 个 Commands：把 workflow 变成一条入口

Commands 是“入口层”。新手不用背完整 prompt，只要知道自己要规划、调试、审查、验证，就能通过 slash command 进入正确流程。

Command	用途
/plan	进入规划模式，先 brainstorm 再写可执行计划。
/tdd	用严格 TDD 实现功能或修 bug。
/verify	运行真实命令验证 claim。
/review	审查当前 diff 或指定路径。
/security-scan	安全扫描：OWASP、secrets、注入、鉴权。
/debug	系统性调试：复现、隔离、修复、验证。
/refactor	安全重构，先看测试覆盖。
/worktree	创建隔离 git worktree。
/deep-research	多源深度研究并综合报告。
/ship	分支收尾：验证后选择 merge/PR/keep/discard。
/onboard	新代码库 onboarding，识别结构、栈和关键文件。
/think	用 sequential-thinking 做复杂推理。
/explain	解释代码、错误或概念。
/gen-tests	为文件或函数生成测试。
/changelog	根据 git 历史生成 Keep a Changelog 条目。
/eli5	用类比给小白解释技术概念。
/regex	按需求编写正则和测试用例。
/cleanup	清理当前 diff 中死代码、未用 import、TODO、console.log。
/deps-audit	审计依赖过期、漏洞、未使用。
/skills	列出可用 skills。
/memory-search	搜索 auto-memory。
/mcp-status	检查 MCP 配置状态。
/backup	备份 Claude Code 配置为 zip。
/agents	列出可用 agents。
/doctor	诊断 Claude Code 配置健康。

5.7 Hooks：把安全、质量和上下文自动化

Hooks 是“安全网”。模型可能忘记规则，但 hook 在工具调用前后执行，能把关键边界从“靠自觉”变成“靠机制”。仓库中 hook 脚本如下：

Hook	注册/用途位置	作用
block-dangerous.ps1	PreToolUse Bash/PowerShell	拦截 git reset --hard、force push、--no-verify、系统关机/格式化等危险命令。

protect-secrets.ps1	PreToolUse Read/Edit/Write/Bash/PowerShell	保护 .env、SSH private key、AWS credentials、auth.json、token 等敏感文件。
protect-linter-configs.ps1	PreToolUse Edit/Write/MultiEdit	防止为了通过检查而修改 ESLint、Prettier、tsconfig 等质量门配置。
auto-format.ps1	PostToolUse Edit/Write	编辑后自动格式化，减少风格噪音。
typecheck.ps1	PostToolUse Edit/Write async	编辑后异步类型检查，尽早发现破坏。
command-log.ps1	PostToolUse Bash/PowerShell	记录命令审计日志，方便追溯。
session-start-context.ps1	SessionStart	注入 git 状态、最近提交、可用 skills/agents/MCPs。
smart-context.ps1	UserPromptSubmit	根据用户 prompt 提示合适 skill 和注意事项。
notify.ps1	Notification	桌面通知。
stop-sound.ps1	Stop	任务完成提示音。
precompact-backup.ps1	PreCompact	压缩上下文前备份 transcript。
subagent-stop.ps1	SubagentStop	记录 subagent 结束事件。
statusline.ps1	statusLine	每 10 秒刷新状态行，显示项目、分支、模型等上下文。
backup-config.ps1	命令辅助	供 /backup 调用，备份配置。
mcp-doctor.ps1	诊断辅助	检查 MCP 配置和依赖。
post-compact-restore.ps1	压缩辅助	压缩后恢复或提示上下文。

5.8 settings.json: 生命周期编排核心

`settings.template.json` 是 Claude Code 侧的生命周期编排模板。安装器不会简单复制固定 `settings.json`，而是渲染占位符：`{{CLAUDE_HOME}}`、`{{ANTHROPIC_AUTH_TOKEN}}`、`{{ANTHROPIC_BASE_URL}}`、`{{ANTHROPIC_MODEL}}`、`{{TIMEZONE}}` 等。

5.8.1 环境变量设计

变量	说明
<code>ANTHROPIC_AUTH_TOKEN</code>	只有用户显式传入 token 时才写入；默认删除，避免安装器保存凭证。
<code>ANTHROPIC_BASE_URL</code>	只有用户显式指定 Base URL 时写入；通常推荐由 cc-switch 管理。
<code>ANTHROPIC_MODEL</code>	可选模型钉住，用于中转站或脚本化安装。
<code>ANTHROPIC_DEFAULT_HAIKU_MODEL</code>	与 <code>ANTHROPIC_MODEL</code> 同步，兼容后台轻量模型调用。

EDITOR	默认 code -wait，让编辑操作进入 VS Code。
CLAUDE_HOOKS_LOG_DIR	指向 ~/.claude/logs，供 hooks 写审计和诊断日志。
CLAUDE_DISABLED_HOOKS	保留禁用 hook 的开关，便于临时排障。

5.8.2 statusLine 与 permissions

`statusLine` 每 10 秒执行 `hooks/statusline.ps1`。它让会话底部能显示项目、分支、模型或其他上下文。`permissions.deny` 默认拒绝高危 PowerShell 命令：`Format-Volume`、`Stop-Computer`、`Restart-Computer`、`Clear-RecycleBin`、`Remove-PSDrive`。这属于防御纵深：即使模型误判，权限层和 hook 层也会先拦一次。

5.8.3 生命周期注册矩阵

事件	Matcher	脚本	效果
PreToolUse	Bash PowerShell	block-dangerous.ps1	命令执行前拦截危险 shell/Git/系统操作。
PreToolUse	Read Edit Write Bash PowerShell	protect-secrets.ps1	读取或修改前保护密钥类文件。
PreToolUse	Edit Write MultiEdit	protect-linter-configs.ps1	防止通过改 linter 配置绕过质量检查。
PostToolUse	Edit Write	auto-format.ps1	编辑后自动格式化。
PostToolUse	Edit Write	typecheck.ps1	编辑后异步类型检查。
PostToolUse	Bash PowerShell	command-log.ps1	记录命令审计。
SessionStart	-	session-start-context.ps1	会话开始注入 git 状态和项目上下文。
UserPromptSubmit	-	smart-context.ps1	根据用户 prompt 提醒合适 skill。
Notification	-	notify.ps1	桌面通知。
Stop	-	stop-sound.ps1	结束提示音。
PreCompact	-	precompact-backup.ps1	上下文压缩前备份 transcript。
SubagentStop	-	subagent-stop.ps1	记录 subagent 停止。

5.9 为什么这些组合起来才是 Strongest

单独的 `CLAUDE.md` 只是规则；单独的 skill 只是方法；单独的 agent 只是角色；单独的 hook 只是拦截器；单独的 MCP 只是工具。Claude Code Codex Strongest 的强点，是把它们串成闭环：用户提出需求后，`smart-context` 提醒方法，`skills` 定义流程，`commands` 提供入口，`agents` 分担复杂任务，`hooks` 防止越界，MCP 补充外部能力，`settings.json` 负责注册，`doctor/verify` 给出证据，Codex 作为独立第二栈补充执行与对照能力。

这就是第五章要论证的重点：Strongest 不是“大而全”的堆料，而是“每一层都有职责，每一层都能被安装器部署，每一层都能被验证”的工程化系统。

第六章 Codex 配置

6.1 目录与模板

Codex 的运行目录是 `~/.codex`。仓库内的 `.codex/` 是安全模板目录，安装器只复制这些文件：

- `AGENTS.md`: Codex 可读的工作规则入口；
- `config.toml`: Codex 基础配置；
- `.gitignore`: 防止 `auth`、`token`、`runtime` 状态被提交；
- `README.md`: 模板说明。

6.2 不会复制什么

注意

安装器不会复制或生成 `auth.json`、`.credentials.json`、`token`、`key`、`sessions`、`logs`、`cache`、`runtime` 等状态。用户必须通过 `codex login` 自己建立 Codex 登录态。

6.3 Codex 登录

安装结束后，运行：

```
codex login
```

登录后可以验证：

```
codex --version
codex doctor --summary --ascii --no-color
```

在一次真实验证中，本机输出了：

```
codex-cli 0.139.0
Codex Doctor v0.139.0 ...
15 ok | 1 idle | 2 notes | 2 warn | 0 fail degraded
```

其中 `WebSocket timeout warning` 不等于配置失败；`doctor` 明确显示 `HTTPS fallback` 可能仍可用，且 `exit code` 为 0。

第七章 MCP Servers

安装器会把 8 个 MCP server 写入 ~/.claude.json:

MCP	用途
sequential-thinking	多步推理与复杂决策。
context7	实时库文档查询。
deepwiki	公开 GitHub repo 文档问答。
playwright	浏览器自动化、截图、UI 验证。
memory	知识图谱式记忆。
fetch	HTTP 抓取。
time	时区转换，默认 Asia/Shanghai。
git-mcp	Git 操作。

7.1 Windows 与 macOS 的命令差异

Windows 模板将 stdio MCP 命令包在 cmd /c 后面，例如 cmd /c npx -y ... 或 cmd /c uvx ...。这样可以提升 Windows 下 npm、uvx、PATH 解析的稳定性。macOS 模板则直接调用 npx 或 uvx。

MCP	Windows	macOS
sequential-thinking	cmd /c npx -y @modelcontextprotocol/server-sequential-thinking	npx -y @modelcontextprotocol/server-sequential-thinking
playwright	cmd /c npx -y @playwright/mcp@latest --headless	npx -y @playwright/mcp@latest --headless
memory	cmd /c npx -y @modelcontextprotocol/server-memory	npx -y @modelcontextprotocol/server-memory
fetch	cmd /c uvx mcp-server-fetch	uvx mcp-server-fetch
time	cmd /c uvx mcp-server-time --local-timezone <TZ>	uvx mcp-server-time --local-timezone <TZ>
git-mcp	cmd /c uvx mcp-server-git	uvx mcp-server-git

7.2 写入策略

安装器会把模板里的 `{{TIMEZONE}}` 替换为用户传入的 IANA 时区，默认 `Asia/Shanghai`。随后覆盖写入 `~/.claude.json` 的 `mcpServers` 属性；如果 `hasCompletedOnboarding` 缺失，则设为 `true`，避免 `fresh/reset` 用户重复触发 `onboarding`。

第八章 API Key 与模型配置

8.1 Claude Code 侧：cc-switch

本项目取消了安装器中的 API key 弹窗。安装完成后，cc-switch 会打开，用户在其中添加 provider：

1. 点击 Add Provider；
2. 输入 API key 与 Base URL；
3. 选择 Claude/Anthropic 预设，或使用中转站配置；
4. 点击 Enable；
5. cc-switch 写入 Claude Code 所需配置。

8.2 Codex 侧：codex login

Codex 登录不通过安装器写入，必须手动执行：

```
codex login
```

8.3 模型名与中转站

对于 Claude Code，安装器支持：

- `-ApiToken / --token`：脚本化写入 token；
- `-BaseUrl / --url`：指定 API base URL；
- `-Model / --model`：同时设置 `ANTHROPIC_MODEL` 与 `ANTHROPIC_DEFAULT_HAIKU_MODEL`；
- `-NoCcSwitch / --no-cc-switch`：跳过 cc-switch。

第九章 常用操作

9.1 验证安装

Windows 下可运行:

```
claude --version
codex --version
codex doctor --summary --ascii --no-color
code --list-extensions | findstr /i "anthropic.claude-code openai.chatgpt"
```

Git Bash / macOS 下可运行:

```
claude --version
codex --version
codex doctor --summary --ascii --no-color
code --list-extensions | grep -Ei '^(anthropic\.claude-code|openai\.chatgpt)$'
```

9.2 Reset 干净重装

Windows:

```
.\install\install-windows.ps1 -Reset
```

macOS:

```
bash install/install-macos.sh --reset
```

9.3 非交互安装

Windows:

```
.\install\install-windows.ps1 -NonInteractive -Force
```

macOS:

```
bash install/install-macos.sh --non-interactive --force
```


第十章 排错指南

10.1 GitHub 下载失败

现象：bootstrap 下载 zip/tar.gz 失败。原因通常是网络、GitHub 访问、代理或证书问题。

处理：

- 确认 VPN / 代理；
- 直接访问 Releases 页面下载入口脚本；
- clone 仓库后本地运行安装器。

10.2 winget msstore 证书错误

曾观察到：

```
0x8a15005e : 服务器证书与任何预期值都不匹配
```

若包在 winget 源存在，可显式指定：

```
winget install -e --id <Package.Id> --source winget
```

10.3 VS Code 扩展安装出现 Node DeprecationWarning

VS Code CLI 可能输出 DEP0169 warning。Windows 安装器已修复：不再把 stderr warning 当成安装失败，而是检查 \$LASTEXITCODE。

10.4 正在 Claude Code 内运行安装器导致.claude 被占用

现象：备份 ~/.claude 时提示“另一个程序正在使用此文件”。

当前 Windows 安装器已修复：

- 先尝试 Move-Item 整目录备份；
- 若失败，使用 robocopy 复制备份；
- 不删除正在使用的目录；
- 继续 overlay 部署。

10.5 Codex doctor WebSocket warning

若看到:

```
Responses WebSocket timed out; HTTPS fallback may still work
```

这通常不是安装失败。只要 doctor exit code 为 0，且没有 fail 项，就可以继续使用；若实际请求失败，再检查代理、VPN、防火墙、DNS、证书和 WebSocket 策略。

10.6 codex 命令找不到

处理顺序:

1. 重新打开终端，让 PATH 刷新；
2. 检查 npm 全局 bin 是否在 PATH；
3. 运行 `npm install -g @openai/codex`；
4. 再运行 `codex --version`。

第十一章 安全模型

11.1 不迁移凭证

项目的核心安全原则：安装器可以部署配置，但不能替用户迁移或生成凭证。

禁止复制或提交：

- `auth.json`
- `.credentials.json`
- `*.token`
- `*.key`
- `.env` 与 `.env.*`
- `sessions`、`history`、`logs`、`tmp`、`cache`、`runtime`、`backups`

11.2 Git 安全

默认规则：

- 不使用 `--no-verify`;
- 不 `force push`，除非用户明确要求;
- 不提交 `secrets`;
- `destructive operations` 前必须确认。

第十二章 维护与发布

12.1 仓库重命名

项目仓库已重命名为:

```
liujiarui0918/claude-code-codex-strongest
```

本地 remote 应为:

```
origin https://github.com/liujiarui0918/claude-code-codex-strongest.git
```

12.2 提交前检查

建议每次发布前运行:

```
git status --short
powershell -NoProfile -ExecutionPolicy Bypass -Command "$p='install/install-windows.ps1';
    $tokens=$null; $errors=$null; [System.Management.Automation.Language.Parser]::
    ParseFile($p,[ref]$tokens,[ref]$errors) | Out-Null; if($errors.Count){ exit 1 }"
bash -n install/install-macos.sh
bash -n install-macos.command
```

12.3 发布检查表

1. README 链接是否指向新仓库;
2. bootstrap 是否指向新仓库;
3. Windows 与 macOS installer 是否都包含 Claude + Codex;
4. .codex/.gitignore 是否保护 auth/runtime;
5. 真实运行 Windows installer 是否 exit 0;
6. codex --version 是否可用;
7. code --list-extensions 是否包含两个核心扩展;
8. GitHub release 附件是否包含最新 bat/command。

12.4 Release 与 CI 边界

当前仓库没有 `.github/workflows` 自动化发布配置。对外分发主要依赖手动创建 GitHub Release，并把最新的 `install-windows.bat` 与 `install-macos.command` 作为附件上传。双击入口本身并不包含完整安装逻辑，而是下载 GitHub 上对应 ref 的仓库压缩包再调用仓库内安装器。

12.5 容易写错或理解错的点

- 不要说安装器会弹 API key 输入框；当前设计是安装后打开 `cc-switch`，由用户在 `cc-switch` 里配置 Claude provider。
- 不要说安装器会复制 Codex 认证文件；Codex 认证必须由 `codex login` 创建。
- 不要把 `hooks` 文件夹里的脚本总数等同于注册生命周期 hook 数；注册生命周期 hook 是 12 个，目录里还有辅助脚本。
- 不要把 `/mcp-status` 写成实时连接测试；它只检查配置，实时状态用 Claude Code 内置 `/mcp`。
- 不要说 Windows 安装 PowerShell 7；Windows 使用自带 PowerShell 5.1 即可，macOS 才会安装 PowerShell 7。

第十三章 附录：关键命令速查

13.1 安装

```
# Windows
irm https://raw.githubusercontent.com/liujiarui0918/claude-code-codex-strongest/main/
install/bootstrap.ps1 | iex

# macOS / Linux
curl -fsSL https://raw.githubusercontent.com/liujiarui0918/claude-code-codex-strongest/
main/install/bootstrap.sh | bash
```

13.2 验证

```
claude --version
codex --version
codex doctor --summary --ascii --no-color
code --list-extensions
```

13.3 登录

```
# Claude Code: use cc-switch GUI
# Codex:
codex login
```

